



Licenciatura Engenharia Informática e Multimédia
Instituto Superior de Engenharia de Lisboa
Ano letivo 2023/2024

Produção de Conteúdos Multimédia
Relatório: Trabalho Final - Image Gallery

Turma: 31D

Nome: Miguel Alcobia Número: 50746

Nome: Fábio Pestana Número: 50756

Professor: Pedro Albuquerque Santos

Data: 11 de Janeiro de 2024

Índice

Lista de figuras.....	II
Abreviaturas e símbolos.....	III
Introdução	1
Image Search.....	2
1. 1 - Protótipo do <i>site</i>	2
1.2 - Implementação da interface.....	2
1.2.1 - Ecrã de Loading animado.....	3
1.2.2 - Dark Mode com Animação Sol/Lua (Icons)	4
1.2.2 - Música de fundo	4
1. 3 - Implementação do processamento de imagens.....	4
1.3.1 - Leitura do ficheiro JSON.....	4
1.3.2 - Disposição em <i>grid</i>	5
1.3.3 - Pesquisa por palavra-chave.....	5
1.3.4 - Pesquisa por cor (Histograma)	5
1.3.5 - Pesquisa por imagem semelhante	6
Comentários e Conclusões	7
Bibliografia	8

Lista de figuras

Figura 1 - Protótipo do *site*2

Figura 2 - Imagem do ícone em grande resolução 3

Figura 3 - Comparação entre Light Mode e Dark Mode 4

Abreviaturas e símbolos

Lista de abreviaturas

API Application Programming Interface

Introdução

Neste trabalho, os alunos terão como objetivo desenvolver uma aplicação multimédia em HTML5, incluindo API's JavaScript, para pesquisa e visualização de fotos digitais de uma coleção de fotografias.

Analisaremos todas as etapas do processo de desenvolvimento da aplicação, desde do desenho protótipo até à implementação de cada uma das *features*.

Image Search

1. 1 - Protótipo do *site*

Ao receber o enunciado, o grupo analisou o que era pedido e elaborou o seguinte esboço para o *layout* do site:

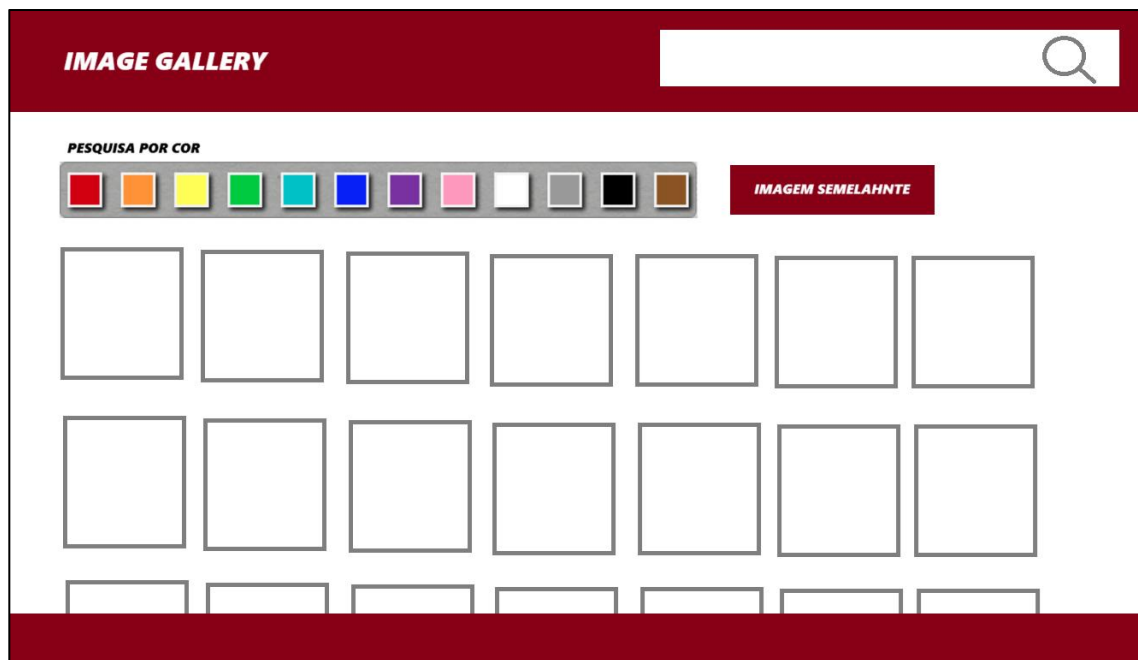


Figura 1 - Protótipo do *site*

Neste esboço já temos as imagens apresentadas numa *grid*. A pesquisa por cor apresenta a lista de 12 cores solicitada no enunciado. O botão “Imagem Semelhante” (que durante o desenvolvimento da aplicação, tornou-se numa *checkbox*) serve para quando o *user* clicar numa imagem, tendo a *checkbox* marcada, serão apresentadas no *grid* imagens semelhantes à que fora selecionada.

Por fim, no canto superior direito da página é apresentada ao *user* uma barra de pesquisa, onde deve colocar as palavras-chave das imagens que pretende encontrar.

1.2 - Implementação da interface.

A implementação foi realizada da seguinte forma: Primeiro realizou-se a interface em HTML5 e CSS e depois fez-se alguns ajustes conforme a necessidade.

Mudou-se o ícone do site para a seguinte imagem, que foi composta no *site* Canvas:

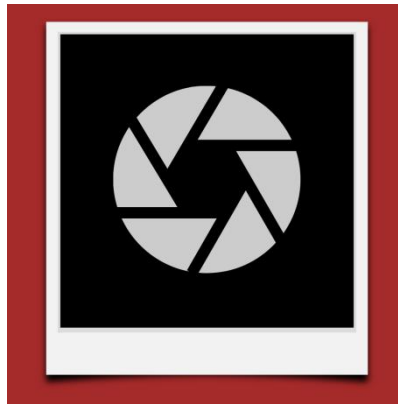


Figura 2 - Imagem do ícone em grande resolução

1.2.1 - Ecrã de Loading animado

Como as imagens do ficheiro JSON demoravam a carregar e o site ficava inutilizável até o final do processamento. Para “entreter” o *user* criou-se, já no final do projeto, um ecrã de loading que se sobrepõe à interface do site e desaparece quando as imagens são totalmente processadas.

Para isso criou-se uma nova *section* no ficheiro HTML:

```
<section id="loadScreen" class="mt-4">
  <div class="loading-wrapper">
    <div class="center">
      <div class="ring"></div>
      <span>Loading...</span>
    </div>
  </div>
</section>
```

Em CSS fez-se uma animação para que um feixe de cor andasse às voltas no círculo de Loading e nesse processo o feixe apresenta uma escala de cinza:

```
.ring {
  position: absolute;
  width: 200px;
  height: 200px;
  border-radius: 50%;
  animation: ring 2s linear infinite;
}

(...)

@keyframes ring {
  0%{
    transform: rotate(0deg);
    box-shadow: 1px 5px 2px white;
  }
  25%{
    transform: rotate(90deg);
    box-shadow: 1px 5px 2px grey;
  }
  50%{
    transform: rotate(180deg);
    box-shadow: 1px 5px 2px black;
  }
  75%{
    transform: rotate(270deg);
    box-shadow: 1px 5px 2px grey;
  }
  100%{
    transform: rotate(360deg);
    box-shadow: 1px 5px 2px white;
  }
}
```

Com a parte visual do Loading feita, faltava implementar o modo de este desaparecer quando o processamento das imagens terminasse, para tal, criou-se uma a função `fadeOut()` na classe `ISearchEngine` que vai reduzindo a opacidade dos elementos que compõe o ecrã de loading a cada 100 milissegundos até que estes desapareçam por completo. Este método `fadeOut()` é invocado no `imageProcessed()`.

1.2.2 - Dark Mode com Animação Sol/Lua (Icons)

Para poupar os olhos do *user* procedeu-se à implementação de um “dark mode” bastante simples. Criou-se um botão animado (que transita entre uma Lua e um Sol) e a cada mudança no estado do botão as cores dos elementos são trocadas.

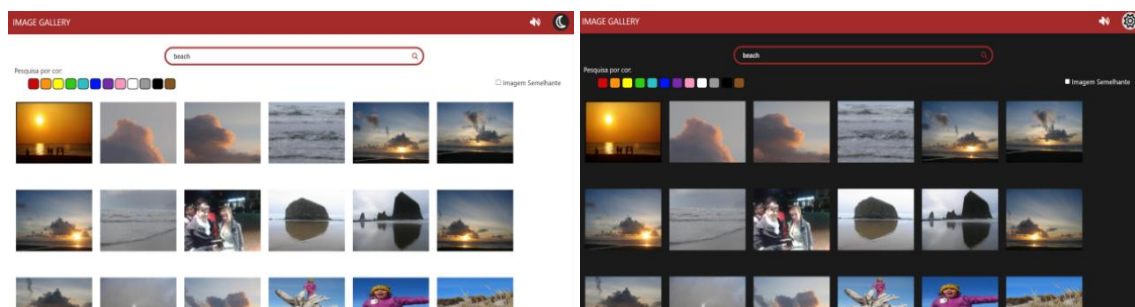


Figura 3 - Comparação entre Light Mode e Dark Mode

1.2.2 - Música de fundo

Na *navebar* é apresentado o ícone (botão) de um altifalante “cortado” que significa que a música de fundo está desligada e ao clicar no ícone, o altifalante é ativado e a música começa a tocar. Isto acontece, porque ao clicar nos altifalantes o método `changeOnOff()` do `ISearchEngine_Manager` é chamado.

1.3 - Implementação do processamento de imagens

1.3.1 - Leitura do ficheiro JSON

Com a interface feita, começou-se a tratar das funções em JavaScript que permitem o funcionamento do sistema de procura de imagens. Para tal, foi necessário fazer a leitura do ficheiro JSON como base de dados. Completou-se a função `databaseProcessing()` da classe `ISearchEngine` onde se cria um array `listOfImages`, que ficará com as imagens que serão processadas.

Depois a função `loadFile()` da classe `DatabaseJSON` é chamada para carregar a informação do arquivo JSON.

Para cada categoria, é feita uma procura para encontrar 30 imagens relacionadas com essa categoria, utilizando a função `search()` da classe `DatabaseJSON`. Esta informação é guardada num array (`procuraCategoria`) e é usada para instanciar uma nova `Picture`, correspondente a cada uma das imagens encontradas.

Em seguida é criado um evento para cada imagem para se saber quando é que ela já foi processada. Por fim, o método `computation()` da classe `Image_Processing` é chamado para fazer o processamento do Objeto `Picture`.

1.3.2 - Disposição em *grid*

A esta altura já era possível ver as fotos a carregar, mas ficavam sobrepostas. Para contornar este problema completou-se a função `gridView()` da classe `ISearchEngine`. Nesta função começamos por limpar o conteúdo do `canvas`, caso contrário ficaríamos com as imagens de umas pesquisas para as outras. Depois, o array `imgCanvas` (que é preenchido ao se realizar a pesquisa) é percorrido para posicionar as várias imagens.

Se a posição em x da imagem mais a largura da mesma, for maior que a largura do `canvas`, a imagem passa para a linha de baixo e o incremento das posições em x volta ao valor inicial. Contudo, se tal não se verificar, as imagens vão sendo desenhadas lado a lado e o valor da posição em x é incrementado.

1.3.3 - Pesquisa por palavra-chave

Ao clicarmos no ícone da lupa na barra de pesquisa, a função `searchKeyWord()` da classe `ISearchEngine_Manager` é chamada e, que por sua vez chama `searchKeywords()` da classe `ISearchEngine`.

Nesta função, a string da categoria pela qual se pesquisou é posta em minúsculas para garantir a boa comparação de strings. Verifica-se se a categoria pesquisada existe na lista de categorias, caso não exista a função é interrompida. Caso exista, procuram-se imagens dessa categoria e adicionam-se ao array `imgCanvas` até este ter 30 imagens.

1.3.4 - Pesquisa por cor (Histograma)

Criaram-se 12 botões, correspondentes às 12 cores propostas no enunciado. Em seguida, no `main()` é criado um evento para cada botão e ao clicar-se nele é chamado o método `searchColor()`, associado à cor do botão selecionado.

Este método funciona de forma semelhante ao `searchKeywords()` da classe

`ISearchEngine`. Contudo, dentro da categoria pesquisada, só serão selecionadas as fotos com a cor do botão selecionado.

Esta cor é procurada nos dados guardados no `LocalStorage`. Na função `createColorDatabaseLS()` da classe `ISearchEngine`, cria-se um objeto `localStorageData` que tem uma propriedade `imagem`. Em seguida percorre-se todas as imagens e se a imagem atual for da categoria atual, ela é adicionada a uma lista chamada `colors`.

As imagens da lista `colors`, são ordenadas pela cor através do `sortByColor()`.

Criam-se objetos JSON que representam as imagens no banco de dados, tendo como classe a cor e o *path* da imagem.

Os objetos JSON criados são adicionados ao `localStorageData.images`. Após o preenchimento do `localStorageData`, o método `save()` da classe `LocalStorageDatabaseJSON` é chamado para salvar os dados no `LocalStorage`.

O `sortByColor()` funciona a partir da comparação dos histogramas das imagens, que são criados no método `computation()` da classe `Image_Processing`, através do método `count_Pixels()`. Neste método são implementadas as fórmulas da distância de Manhattan, apresentadas no enunciado.

1.3.5 - Pesquisa por imagem semelhante

Para pesquisar imagens semelhantes a uma outra, fez-se o seguinte: adicionou-se um evento ao `canvas`, que quando se clica numa imagem o método `selectImg()` da classe `ISearchEngine_Manager` é chamado, que por sua vez chama o método `searchISimilarity()` que vai procurar imagens semelhantes à selecionada, através da informação guardada no `LocalStorage`.

Comentários e Conclusões

Neste trabalho enfrentámos dificuldades no início do mesmo, pois é sempre complicado pegar em código de terceiros e entender a sua lógica, mas faz parte. Além disso, a implementação da pesquisa de imagens por similaridade não ficou bem, mas desconhecemos o motivo do problema.

Foi engraçado implementar um recurso que usamos no dia a dia, como é o caso de uma galeria de imagens, e entender a lógica por detrás da mesma.

Bibliografia

PowerPoint - "*Projeto Final*" - disponibilizado pelo professor

PowerPoints das aulas, disponibilizado pelo professor